

## Тема 11 – Структури от данни.

1. Структури от данни.
2. Систематизация на основните структури от данни.
3. Дефиниране на абстрактен тип данни (АТД).
4. Статични АТД.

## Тема 11 – Структури от данни.

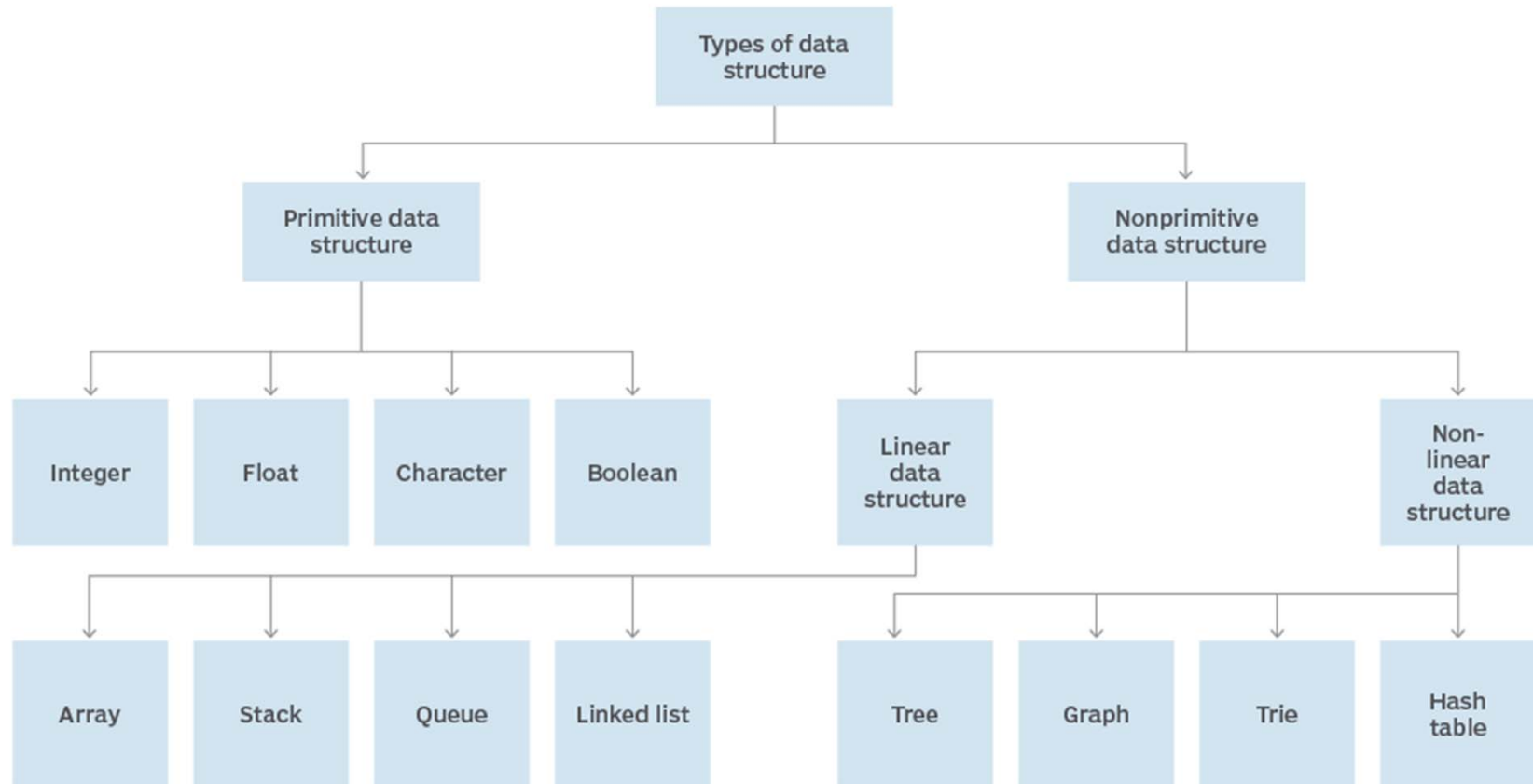
Абстрактен тип данни (АТД или ADT) е тип (или клас) за обекти, чието поведение се определя от набор от стойности и набор от операции.

Дефиницията на ADT само споменава какви операции трябва да бъдат изпълнени, но не и как тези операции ще бъдат изпълнени.

Не се уточнява как данните ще бъдат организирани в паметта и какви алгоритми ще се използват за изпълнение на операциите.

Нарича се „абстрактен“, защото дава независим от изпълнението изглед. Процесът на предоставяне само на най-важното и скриване на детайлите е известен като абстракция.

# Data structure hierarchy



## Тема 11 – Структури от данни.

Всяка структурата на данните има свой специализиран формат за организиране, обработка, извличане и съхраняване на данни. Има примитивни и не примитивни типове структури на данни, всички предназначени да подредят данните така, че да отговарят на конкретна цел. Структурите на данни улесняват достъпа на потребителите и работата с данните, от които се нуждаят, по подходящи начини. Най-важното е, че структурите на данни рамкират организацията на информацията, така че машините и хората да могат да я разберат и използват коректно.

В компютърните науки и компютърното програмиране структура на данни може да бъде избрана или проектирана да съхранява данни с цел обработката им с различни алгоритми. В някои случаи основните операции на алгоритъма са тясно свързани с дизайна на структурата на данните. Всяка структура на данни съдържа информация за стойностите на данните, връзките между данните и – в някои случаи – функции, които могат да бъдат приложени към работа с данните.

## Тема 11 – Структури от данни.

В обектно-ориентиран език за програмиране структурата на данните и свързаните с нея методи са обвързани заедно като част от дефиниция на клас.

В необектно-ориентираните езици може да има функции, дефинирани за работа със структурата на данните, но те технически не са част от структурата на данните.

## Тема 11 – Структури от данни.

Типичните базови (примитивни) типове данни, като цели числа или стойности с плаваща запетая, които са налични в повечето езици за компютърно програмиране, обикновено са недостатъчни за улавяне на логическото намерение за обработка и използване на данни.

Структурите от непримитивни данни са градивните елементи за по-сложни приложения, като в зависимост от спецификите на задачата и на самите данни, те могат според организацията си да бъдат линейни и нелинейни структури на данни.

## Тема 11 – Структури от данни.

Важно е не само да умеете да използвате различни структури от данни, но също така е важно да изберете правилната структура от данни за всяка задача. Изборът на неподходяща структура на данните може да доведе до бавно време на изпълнение или утежнен код. Фактори, които трябва да имате предвид при избора на структура от данни, са следните:

- \* Какъв вид информация ще се съхранява?
- \* Как ще се използва тази информация?
- \* Къде трябва да се съхраняват данните, след като са създадени?
- \* Кой е най-добрият начин за организиране на данните?
- \* Какви аспекти на управлението на паметта и процедурите за съхраняване, характерни за специфичната памет, трябва да се имат предвид

# Тема 11 – Структури от данни.

Структурите от данни се класифицират по техните характеристики:

1. **Линейни или нелинейни.** Тази характеристика описва дали елементите с данни са подредени в последователен ред, като например с масив, или в неподредена последователност, като например в граф.
2. **Хомогенни или разнородни.** Тази характеристика описва дали всички елементи от данни в дадено хранилище са от един и същи тип. Един пример е колекция от елементи в масив, дефиниран като структура в С или класова спецификация в Java.
3. **Статични или динамични.** Тази характеристика описва как се компилират структурите от данни. Статичните структури от данни имат фиксирани размери, структури и места в паметта по време на компилиране. Динамичните структури от данни имат размери, структури и места в паметта, които могат да се свиват или разширяват в зависимост от употребата.



## Тема 11 – Структури от данни.

**Масив.** Масивът съхранява колекция от елементи в съседни места в паметта. Елементите, които са от един и същи тип, се съхраняват заедно, така че позицията на всеки елемент може да бъде изчислена или извлечена лесно чрез индекс. В различните езици за програмиране масивите са изградени по различни начини.

1. Масивът може да бъде **едномерен, многомерен** или **масив от масиви**.
2. Масивите са базирани на нулево индексване – това означава, че в масив с  $N$  елемента, първият елемент ще е с индекс нула, а последният с индекс  $N-1$ .
3. Елементите на масива могат да бъдат от всякакъв тип, включително от тип масив.
4. Стойността по подразбиране на елементите от числен тип е нула, за референтните типове (указател) е `null`, а за булевите типове е `false`. При масив от масиви елементите са от референтен тип и по подразбиране са `null`.
5. **Редът на елементите и дължината на масива са фиксирани.**

## Тема 11 – Структури от данни.

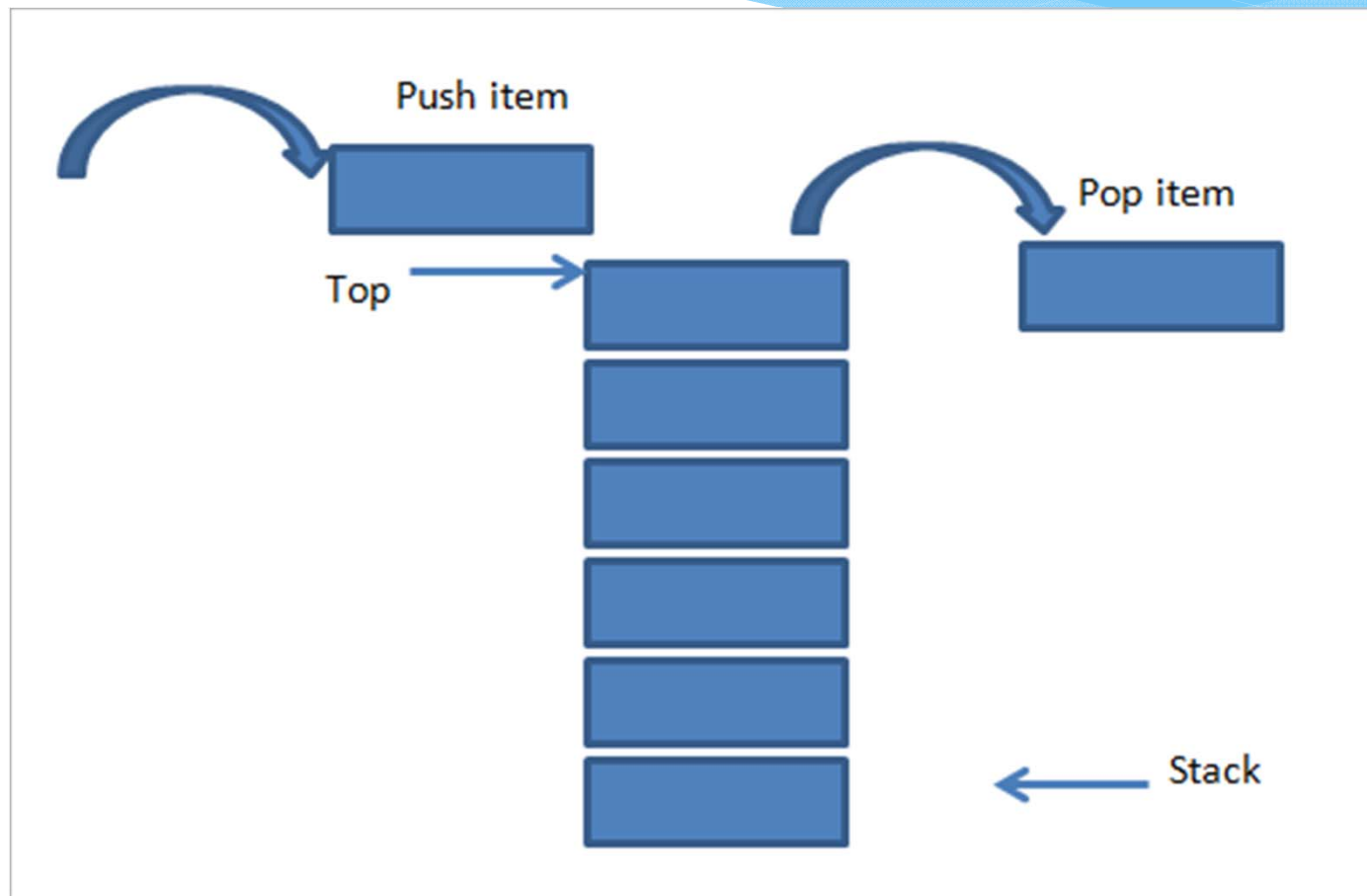
**Стек.** Стекът е линейна структура от данни в информатиката, в която обработката на информация става само от едната страна наречена връх. Дъното не е и не трябва да е достъпно. Стековете са базирани на принципа „последен влязъл пръв излязъл“ (LIFO).

Стекът е абстрактен тип данни, който служи като колекция от елементи с две основни операции:

**Push**, който добавя елемент към колекцията и

**Pop**, което премахва най-скоро добавения елемент, който все още не е премахнат.

## Тема 11 – Структури от данни.



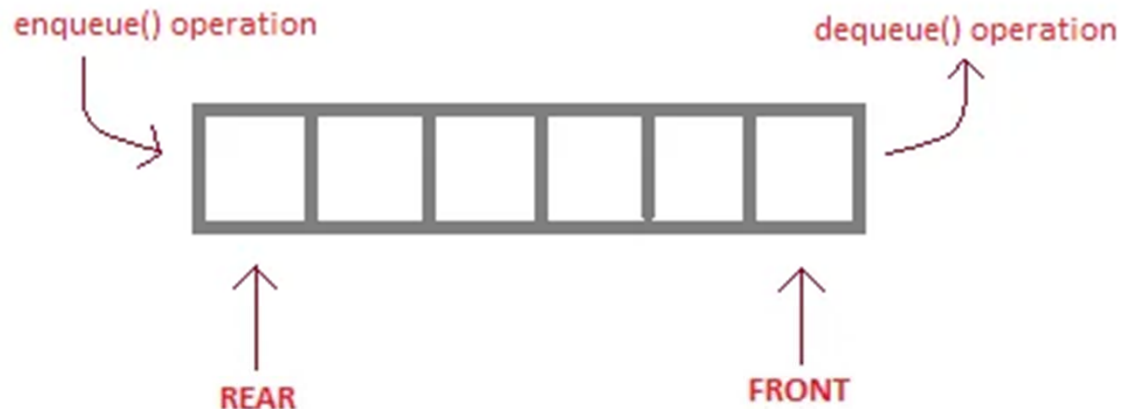
## Тема 11 – Структури от данни.

**Опашка.** Опашката се дефинира като линейна структура от данни, която е отворена в двата края и операциите се изпълняват в реда първи влязъл, първи излязъл - FIFO.

Опашката се дефинира като списък, в който всички добавяния към списъка се правят в единия край, а всички изтривания от списъка се правят в другия край.

Позицията на записа в опашката, готова за обслужване, тоест първият запис, който ще бъде премахнат от опашката, се нарича предната част на опашката (head of the queue), по подобен начин позицията на последния запис в опашката, тоест най-скоро добавеният, се нарича задната част (the tail) на опашката.

## Тема 11 – Структури от данни.



enqueue( ) is the operation for adding an element into Queue.

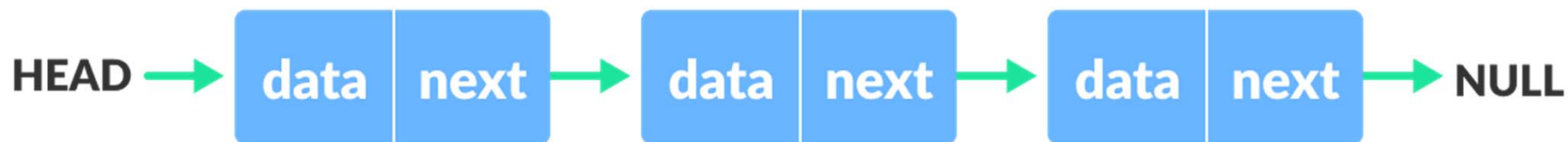
dequeue( ) is the operation for removing an element from Queue .

### QUEUE DATA STRUCTURE

И стековете, и опашките могат да бъдат представени в масив, тоест тези две структури могат да се реализира с помощта на масив.

## Тема 11 – Структури от данни.

**Свързан списък.** Свързаният списък съхранява колекция от елементи в линеен ред. Всеки елемент или възел в свързан списък съдържа елемент от данни, както и препратка или връзка към следващия елемент в списъка.



## Тема 11 – Структури от данни.

**Свързаният списък е линейна колекция от елементи от данни, чийто ред не се дава от физическото им разположение в паметта. Вместо това всеки елемент сочи към следващия.**

Свързаният списък е най-търсената структура от данни, когато става въпрос за обработка на динамични елементи от данни. Свързаният списък се състои от елементи от данни, известни като възли (node).

Всеки възел се състои от две полета: едното поле съдържа данни, а във второто поле има адрес, който поддържа препратка към следващия възел.

## Тема 11 – Структури от данни.

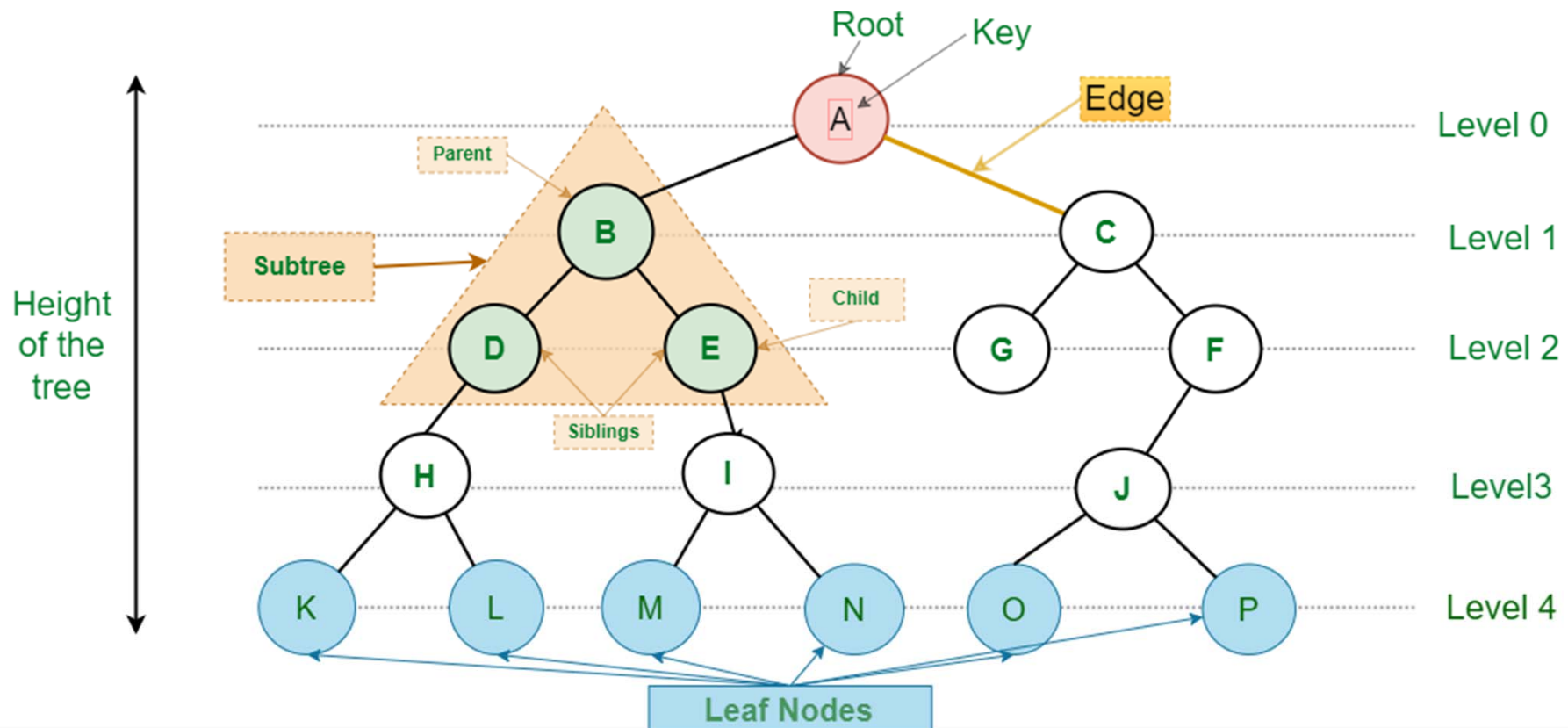
**Структура на данни дърво.** Едно дърво съхранява колекция от елементи по абстрактен, йерархичен начин. Всеки възел е свързан с ключова стойност, като родителските възли са свързани с дъщерни възли - или подвъзли. Има един коренов възел, който е предшественик на всички възли в дървото.

Дървото в програмирането е рекурсивна структура от данни, която се състои от върхове, които са свързани помежду си с ребра. За дърветата са в сила твърденията: Всеки връх може да има нула или повече преки наследници. Всеки връх има най-много един родител.



# Тема 11 – Структури от данни.

## Tree Data Structure



## Тема 11 – Структури от данни.

**Структура на данни дърво.** Едно дърво съхранява колекция от елементи по абстрактен, йерархичен начин. Всеки възел е свързан с ключова стойност, като родителските възли са свързани с дъщерни възли - или подвъзли. Има един коренов възел, който е предшественик на всички възли в дървото.

Дървото в програмирането е рекурсивна структура от данни, която се състои от върхове, които са свързани помежду си с ребра. За дърветата са в сила твърденията: Всеки връх може да има нула или повече преки наследници. Всеки връх има най-много един родител.

## Тема 11 – Структури от данни.

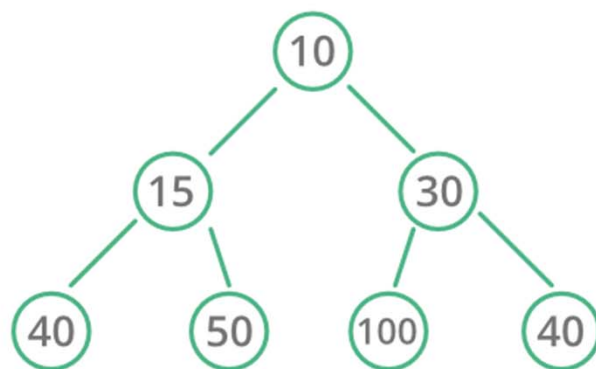
Дърво, в което броят на наследниците на върховете е 0, 1 или 2 се нарича **двоично дърво**. Преките наследници на всеки връх са **два** в този случай, затова единият се нарича **ляв** наследник, а другият – **десен** наследник (може и празно множество). Те, от своя страна, са корени съответно на лявото поддърво и на дясното поддърво на техния родител.

Има няколко основни операции при двоичните дървета: **добавяне, изтриване, търсене, обхождане.**

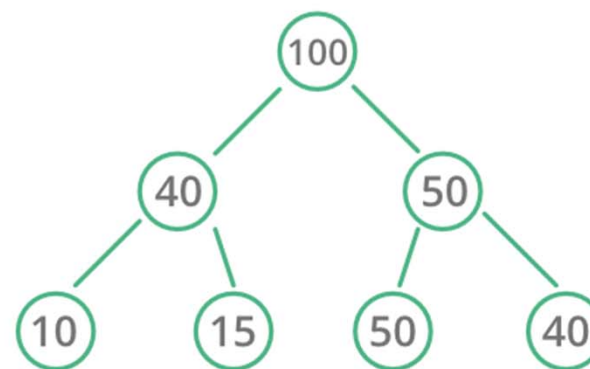
## Тема 11 – Структури от данни.

**Heap** е дървовидна структура, в която асоциираната ключова стойност на всеки родителски възел е по-голяма или равна на ключовите стойности на която и да е от неговите дъщерни ключови стойности.

### Heap Data Structure



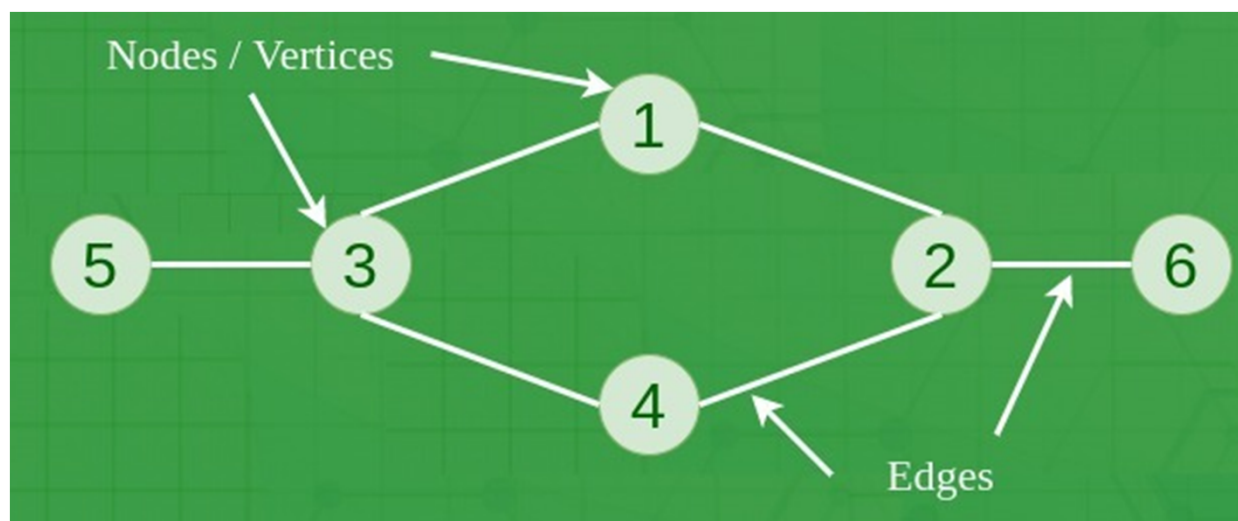
Min Heap



Max Heap

## Тема 11 – Структури от данни.

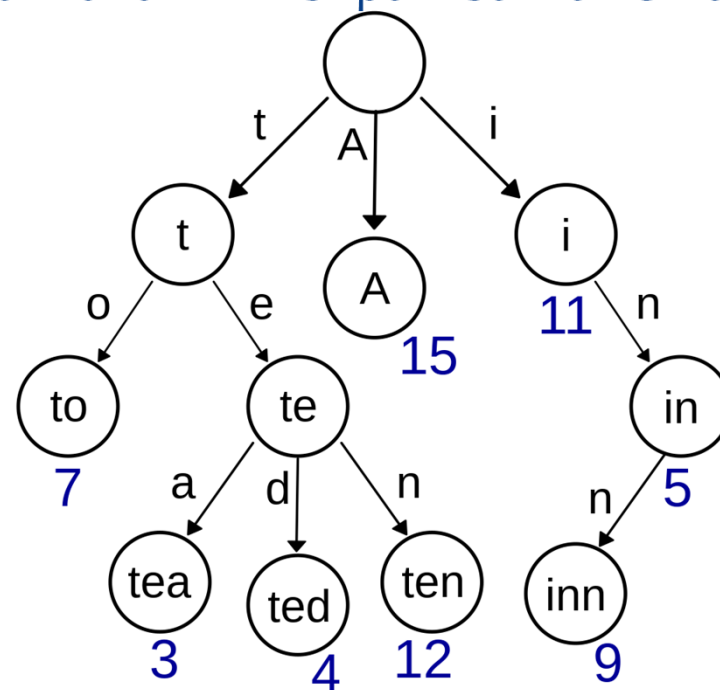
**Граф.** Графът съхранява колекция от елементи по нелинеен начин. Графовете са съставени от краен набор от възли (nodes) или върхове (vertices), и линии, които ги свързват, известни също като ребра или ръбове (**edges**). Те са полезни за представяне на системи от реалния свят като компютърни мрежи и други.



## Тема 11 – Структури от данни.

**Trie** е дървоървовидна структура на данни, цифрово дърво или префиксно дърво, която съхранява низове като елементи от данни, които могат да бъдат организирани във визуален граф. Всички деца на възел имат общ префикс на низа, свързан с този родителски възел, а коренът е свързан с празния низ. Тази задача за съхраняване на данни, достъпни чрез неговия префикс, може да бъде изпълнена по оптимизиран за паметта начин чрез използване на радикално дърво.

Trie структури от данни обикновено се използват в речници с предсказуем текст или автоматично довършване и алгоритми за приблизително съвпадение.



## Тема 11 – Структури от данни.

**Хеш таблица.** Хеш-таблица - известна още като хеш-карта - съхранява колекция от елементи в асоциативен масив, който начертава ключове към стойности. Хеш таблицата използва хеш функция за преобразуване на индекс в масив от **buckets** (кофи), които съдържат желаня елемент от данни.

